



**Universite Hassan II de Casablanca
Faculte des Sciences Ben M'Sick
Departement de Mathematiques et Informatique**

*Memoire de fin d'etude pour l'obtention de la Licence Sciences
Mathematiques et Informatique - Option : Genie Logiciel*

Sujet :

E-STORE

Conception et realisation d'une plateforme e-commerce
full-stack a persistance hybride (SQL + NoSQL)

Encadre par : Pr. ZAHOUR OMAR

Realise par :

Akram BELMOUSSA - Nouhaila BEN SOUMANE

Soutenu en juin 2026, devant le jury :

Pr. Omar ZAHOUR
Pr. [Nom du President]
Pr. [Nom de l'Examineur]

Encadrant
President du jury
Examineur

Dedicace

C'est avec profonde gratitude et sinceres mots, que nous dedions ce modeste travail de fin d'etudes a nos chers **parents**, qui ont sacrifie leur vie pour notre reussite et nous ont eclaire le chemin par leurs conseils judicieux.

Nous esperons qu'un jour, nous serons leur source de fierte et de bonheur. Nous prions Allah de leur preter longue vie en bonne sante.

Nous dedions egalement ce travail a nos **freres et soeurs**, a nos **familles**, a nos **amis**, et a tous nos chers **professeurs**.

Akram & Nouhaila

Remerciements

Au terme de ce projet, il nous est agreable d'exprimer nos sincerres remerciements a Allah le Tout Puissant qui nous a donne la sante et la volonte d'entamer et de terminer ce memoire.

Nous tenons a remercier infiniment toute l'equipe pedagogique de la **Faculte des Sciences Ben M'Sick**, et tout particulierement les enseignants du **departement des Sciences Mathematiques et Informatique**, dont les enseignements ont constitue les fondations sur lesquelles ce travail a pu s'elever.

Nous adressons nos vifs remerciements a notre encadrant, **Pr. ZAHOUR OMAR**, qui a garde un oeil attentif sur le deroulement du projet, en donnant des remarques constructives. Nous le remercions pour sa disponibilite, sa rigueur scientifique et ses conseils avises. Cela a ete un plaisir de travailler sous sa directive.

Enfin, nous adressons nos sincerres remerciements aux **membres du jury** qui ont accepte d'evaluer ce modeste travail, ainsi qu'a nos **familles** et **camarades de promotion** pour leur soutien constant.

Resume

Ce memoire presente la conception et la realisation de **E-Store**, une application e-commerce complete developpee dans le cadre du module Full-Stack. Le projet adopte une architecture rigoureuse en **trois couches techniques** (presentation, logique metier, acces aux donnees) et un decoupage en **cinq domaines fonctionnels** suivant les principes du Domain-Driven Design.

L'application implemente le parcours utilisateur d'un site marchand : inscription, authentication, navigation dans un catalogue, gestion d'un panier, validation transactionnelle de commandes et depot d'avis produits. La particularite technique reside dans la mise en oeuvre d'une **persistance hybride** associant un SGBD relationnel (MySQL ou H2) pour les donnees transactionnelles a un SGBD documentaire (MongoDB) pour les avis utilisateurs.

Le backend repose sur **Spring Boot 3.3** et **Spring Security 6** avec une authentication stateless par jeton **JWT**. Le frontend exploite **Angular 17** en mode standalone avec signals et lazy loading. Une suite de neuf tests unitaires JUnit 5 / Mockito couvre les services metier critiques.

Mots-cles : Spring Boot, Angular, JWT, MongoDB, Architecture en couches, Domain-Driven Design, Persistance hybride, ACID, REST.

Abstract

This thesis presents the design and implementation of **E-Store**, a complete e-commerce application developed within the Full-Stack module. The project adopts a rigorous architecture with **three technical layers** (presentation, business logic, data access) and is split into **five functional domains** following Domain-Driven Design principles.

The application implements the full user journey of an online marketplace: registration, authentication, catalog browsing, cart management, transactional order validation, and product reviews. The key technical highlight lies in implementing a **hybrid persistence** scheme combining a relational DBMS (MySQL or H2) for transactional data with a document-oriented DBMS (MongoDB) for user reviews.

The backend relies on **Spring Boot 3.3** and **Spring Security 6** with stateless authentication via **JWT** tokens. The frontend leverages **Angular 17** in standalone mode with signals and lazy loading. A test suite of nine JUnit 5 / Mockito unit tests covers critical business services.

Keywords: Spring Boot, Angular, JWT, MongoDB, Layered architecture, Domain-Driven Design, Hybrid persistence, ACID, REST.

Liste des figures

Figure 1 : Architecture en trois couches de l'application E-Store	11
Figure 2 : Decoupage en domaines fonctionnels	12
Figure 3 : Modele Conceptuel de Donnees relationnel	14
Figure 4 : Structure d'un document MongoDB	16
Figure 5 : Cycle de vie d'un JWT	29
Figure 6 : Diagramme de sequence du checkout	32
Figure 7 : Capture d'ecran - Page d'accueil du catalogue	44
Figure 8 : Capture d'ecran - Fiche produit	45
Figure 9 : Capture d'ecran - Panier utilisateur	46
Figure 10 : Capture d'ecran - Historique des commandes	47

Liste des tableaux

Tableau 1 : Cas d'usage fonctionnels couverts	8
Tableau 2 : Comparaison MySQL vs MongoDB	13
Tableau 3 : Endpoints REST principaux	17
Tableau 4 : Liste des entites JPA	23
Tableau 5 : Tests unitaires implementes	37
Tableau 6 : Metriques quantitatives du projet	49
Tableau 7 : Bilan fonctionnel	50

Table des matieres

Introduction generale	1
Chapitre 1 : Contexte et cahier des charges	3
1.1. Contexte academique	3
1.2. Etude du marche e-commerce	4
1.3. Cahier des charges fonctionnel	5
1.4. Cahier des charges non fonctionnel	7
1.5. Methodologie de travail	8
Chapitre 2 : Architecture et conception	10
2.1. Architecture en trois couches	10
2.2. Decoupage en domaines fonctionnels	12
2.3. Modele conceptuel de donnees	14
2.4. Modele documentaire MongoDB	16
2.5. Architecture API REST	17
Chapitre 3 : Choix techniques et justifications	19
3.1. Backend - Java et Spring Boot	19
3.2. Persistance hybride	20
3.3. Frontend - Angular 17 standalone	21
Chapitre 4 : Realisation du backend	23
4.1. Structure du projet	23
4.2. Domaine customer	24
4.3. Domaine catalog	25
4.4. Domaines shopping et billing	26
4.5. Domaine review (MongoDB)	27
4.6. Gestion globale des erreurs	27
Chapitre 5 : Securite de l'application	29
5.1. Hachage BCrypt	29
5.2. Authentification stateless JWT	29
5.3. Autorisation par role	31
5.4. Configuration CORS	32
Chapitre 6 : Realisation du frontend	33
6.1. Structure de l'application	33
6.2. Routing et lazy loading	34
6.3. Intercepteurs et garde	35

Chapitre 7 : Tests et validation	37
7.1. Strategie de tests	37
7.2. Suites de tests	37
7.3. Resultats	38
Chapitre 8 : Deploiement et DevOps	39
8.1. Profils Spring	39
8.2. Conteneurisation Docker	39
8.3. Versionnement Git / GitHub	40
Chapitre 9 : Difficultes rencontrees	41
9.1. Strictness Mockito	41
9.2. LazyInitializationException	41
9.3. Installation Docker Desktop	42
9.4. Encodage des tokens JWT	42
Chapitre 10 : Resultats et perspectives	44
10.1. Bilan fonctionnel	44
10.2. Metriques	49
Conclusion generale	51
Bibliographie	53
Annexes	54

Introduction generale

L'avenement du commerce electronique a profondement remodele les pratiques commerciales mondiales. Selon les estimations recentes, le volume des transactions en ligne au Maroc a franchi le cap des **20 milliards de dirhams** en 2024, avec une croissance annuelle moyenne superieure a 25%. Cette dynamique s'accompagne d'une exigence accrue en termes d'*ergonomie*, de *performance* et de *securite* des plateformes marchandes.

Le module **Full-Stack**, dispense sous la direction du **Pr. Omar ZAHOUR** a la Faculte des Sciences Ben M'Sick, vise a doter les etudiants des competences necessaires a la conception et la realisation d'applications de bout en bout : du modele de donnees jusqu'a l'interface utilisateur, en passant par la logique metier, la securite et le deploiement.

C'est dans ce contexte que s'inscrit le present projet, baptise **E-Store**. Il s'agit d'une mise en pratique exhaustive des enseignements theoriques par la construction d'une application e-commerce complete, integrant les meilleurs standards de l'industrie en 2025 : Spring Boot 3.3, Angular 17, persistance hybride relationnel/documentaire, authentication JWT, tests unitaires automatises et conteneurisation Docker.

Objectifs du projet

Ce projet poursuit cinq objectifs principaux :

- **Maitriser une architecture full-stack moderne** en appliquant un decoupage en couches et en domaines fonctionnels rigoureux.
- **Illustrer la persistance hybride** en faisant cohabiter, au sein d'une meme application, un SGBD relationnel (MySQL) et un SGBD documentaire (MongoDB).
- **Appliquer les bonnes pratiques** de genie logiciel : DTOs, transactions, validation, gestion centralisee des erreurs, tests unitaires.
- **Securiser l'application** par un mecanisme stateless reposant sur Spring Security 6, BCrypt et JWT.
- **Produire une interface utilisateur reactive** avec Angular 17 en mode standalone et Bootstrap 5.

Plan du document

Le present rapport s'articule autour de neuf chapitres :

- Le **chapitre 1** pose le contexte du projet et formalise le cahier des charges.
- Le **chapitre 2** decrit l'architecture generale, le decoupage en couches et en domaines.
- Le **chapitre 3** justifie les choix techniques operes pour chaque brique de la stack.
- Les **chapitres 4, 5 et 6** detaillent respectivement la realisation backend, securite et frontend.
- Le **chapitre 7** expose la strategie de tests et les resultats obtenus.

- Le **chapitre 8** traite du deploiement, de la conteneurisation et de l'integration continue.
- Le **chapitre 9** relate les difficultes rencontrees et les solutions mises en oeuvre.
- Le **chapitre 10** dresse le bilan des resultats et des metriques.
- Une **conclusion generale** trace les perspectives d'evolution.

Chapitre 1

Contexte et cahier des charges

1. Contexte et cahier des charges

1.1. Contexte academique

Ce projet est realise dans le cadre du module **Full-Stack** du departement d'informatique de la Faculte des Sciences Ben M'Sick. Ce module intervient en fin de cursus, apres que les etudiants ont acquis les bases en programmation orientee objet (Java), bases de donnees relationnelles (SQL, normalisation, MERISE), developpement web cote client (HTML, CSS, JavaScript) et genie logiciel (UML, patrons de conception).

L'objectif pedagogique est triple :

- **Integrer** les acquis des modules precedents au sein d'un projet de taille significative.
- **Decouvrir** les frameworks de production utilises dans l'industrie (Spring Boot, Angular).
- **Pratiquer** le travail en binome sur un depot Git partage, en respectant des conventions de nommage et de commit.

1.2. Etude du marche du commerce electronique

Le commerce electronique marocain connait une expansion soutenue. Les plateformes leaders du marche - *Jumia*, *Avito*, *Marjane.ma*, *Decathlon.ma* - partagent un certain nombre de fonctionnalites-types qui constituent l'etat de l'art : authentication utilisateur, navigation dans un catalogue categorise, recherche textuelle et par filtres, gestion d'un panier persistant, processus de commande securise, suivi d'historique, depot et consultation d'avis clients.

E-Store ne pretend evidemment pas rivaliser avec ces geants, mais propose une *maquette pedagogique fonctionnelle* qui en reprend les fondamentaux dans un perimetre maitrisable.

1.3. Cahier des charges fonctionnel

1.3.1. Acteurs

L'application distingue deux profils d'utilisateurs :

- **USER** (utilisateur standard) : peut s'inscrire, se connecter, consulter le catalogue, ajouter au panier, passer commande, consulter son historique et déposer des avis.
- **ADMIN** (administrateur) : dispose, en plus, des droits de creation, modification et suppression sur les categories, les produits et le stock.

1.3.2. Cas d'usage principaux

Le tableau suivant recapitule les cas d'usage fonctionnels couverts par l'application :

Cas d'usage	Acteur	Description
Inscription	Visiteur	Creation d'un compte avec email unique et mot de passe
Connexion	Utilisateur	Authentication email/mot de passe, retour JWT
Profil	USER	Consultation et mise a jour du profil

Catalogue	Tous	Liste paginee, recherche, filtre par categorie
Fiche produit	Tous	Detail (prix, stock, image), liste des avis
Panier	USER	Ajout, modification, suppression, vidage
Commande	USER	Validation transactionnelle (ACID), historique
Avis	USER	Depot (note 1-5 + commentaire), consultation
CRUD admin	ADMIN	Categories, produits, stock

Tableau 1 : Cas d'usage fonctionnels couverts

1.4. Cahier des charges non fonctionnel

1.4.1. Performance

Les pages doivent se charger en moins de **2 secondes** sur connexion nominale (4G ou Wi-Fi). Le bundle initial JavaScript doit rester sous **50 ko** grace au lazy loading.

1.4.2. Securite

- Mots de passe haches en **BCrypt** (jamais en clair).
- Authentification stateless par **JWT signe HMAC-SHA256**, expiration 24 heures.
- Endpoints d'administration proteges par `@PreAuthorize`.
- Validation systematique des entrees (Bean Validation).
- CORS restreint a l'origine localhost:4200 en developpement.

1.4.3. Maintainabilite

- Code conforme aux conventions Java/TypeScript.
- Decoupage en cinq domaines fonctionnels.
- Au moins neuf tests unitaires couvrant les services metier.
- Documentation interne (Javadoc, README, commentaires).

1.4.4. Portabilite

- Demarrage possible sans installation prealable (profil dev H2).
- Demarrage Docker en une commande pour le profil prod.
- Compatibilite multi-OS (Windows, macOS, Linux).

1.5. Methodologie de travail

Nous avons retenu une approche **iterative legere**, inspiree d'Agile mais adaptee au contexte d'un binome etudiant. Le projet a ete decoupe en **quinze etapes incrementales**, chacune se concluant par un commit Git significatif. La repartition des roles a ete la suivante :

- **Akram Belmoussa** - Backend (Spring Boot, JPA, Securite JWT), integration MongoDB, tests unitaires.
- **Nouhaila Ben Soumane** - Frontend (Angular 17, Bootstrap), modelisation des donnees, documentation.
- **Travail commun** - Architecture, design API, debogage, revue de code.

Chapitre 2

Architecture et conception

2. Architecture et conception

2.1. Architecture en trois couches

E-Store adopte une architecture **n-tiers classique** en trois couches, qui presente l'avantage de separer clairement les responsabilites et de faciliter la maintenance.

2.1.1. Couche presentation (frontend)

Cette couche est en charge de l'interaction avec l'utilisateur final. Elle n'a aucune connaissance du modele de persistance et communique avec le backend exclusivement via une **API REST/JSON**. Implementation : *Angular 17 standalone, Bootstrap 5, RxJS, signals*.

2.1.2. Couche logique metier (backend)

C'est le coeur applicatif. Elle recoit les requetes HTTP, valide les entrees, applique les regles de gestion (verification de stock, calcul de totaux, generation de jetons), orchestre les transactions et renvoie des DTOs serialises en JSON. Implementation : *Spring Boot 3.3, Spring Security 6*.

2.1.3. Couche d'accès aux données

Cette couche, masquee derriere les abstractions Spring Data, gere la persistance effective. Deux SGBD coexistent :

- **MySQL 8** (ou H2 en mode dev) pour les donnees strictement relationnelles : utilisateurs, produits, panier, commandes.
- **MongoDB 7** pour les avis, donnees semi-structurees en forte croissance.

Figure 1 : Architecture en trois couches de l'application E-Store

2.2. Decoupage en domaines fonctionnels

Chaque couche est elle-meme decoupee en **cinq domaines fonctionnels** cohesifs, suivant les principes du *Domain-Driven Design*. Un sixieme domaine, *review*, gere la particularite MongoDB. La matrice ci-dessous donne une vue exhaustive du projet :

Domaine	Presentation	Logique metier	Donnees
customer	login.component	AuthService, JwtFilter	users, profiles
catalog	product-list	ProductService	products, categories
inventory	Indicateur stock	InventoryService	inventories
shopping	cart.component	CartService	carts, cart_items
billing	orders.component	OrderService @Transactional	orders, order_items
review	review-form	ReviewService	MongoDB reviews

Figure 2 : Decoupage en domaines fonctionnels

2.3. Modele conceptuel des donnees

Le modele relationnel comporte **neuf entites** reliees par des associations one-to-one, one-to-many et many-to-one :

Entite	Champs principaux	Relations
User	id, firstName, lastName, email, password	1-1 Profile, 1-N Order
Profile	id, phone, address, city, country	1-1 User
Category	id, name, description	1-N Product
Product	id, name, description, price, imageUrl	N-1 Category, 1-1 Inventory
Inventory	id, quantity	1-1 Product
Cart	id, createdAt, updatedAt	N-1 User, 1-N CartItem
CartItem	id, quantity, unitPrice	N-1 Cart, N-1 Product
Order	id, orderDate, totalAmount, status	N-1 User, 1-N OrderItem
OrderItem	id, quantity, unitPrice	N-1 Order, N-1 Product

Figure 3 : Modele Conceptuel de Donnees relationnel

2.4. Modele documentaire MongoDB

La collection *reviews* stocke les avis utilisateurs sous forme de documents JSON. Voici la structure type :

```
{
  "_id":      ObjectId("..."),
  "productId": 12,
  "userId":   5,
  "authorName": "Akram Belmoussa",
  "rating":   5,
  "comment":  "Excellent produit !",
  "createdAt": ISODate("2025-04-22T14:30:00Z")
}
```

Figure 4 : Structure d'un document de la collection reviews

Deux index secondaires sont crees via les annotations *@Indexed* sur les champs *productId* et *userId*, afin d'accelerer les requetes *findByProductId* et *findByUserId*.

2.5. Architecture API REST

Toute communication frontend/backend transite par une API REST. L'API respecte les standards : methodes HTTP appropriees (GET / POST / PUT / DELETE), codes de retour normalises (200, 201, 204, 400, 401, 403, 404, 409), corps JSON encadre par une enveloppe *ApiResponse* standardisee, versioning implicite par le prefixe */api/*.

Verbe	URL	Description	Auth.
POST	/api/auth/register	Inscription	Public
POST	/api/auth/login	Connexion (renvoie JWT)	Public
GET	/api/users/me	Profil utilisateur courant	USER
GET	/api/categories	Liste des categories	Public
GET	/api/products	Catalogue + filtres	Public
GET	/api/cart	Panier courant	USER
POST	/api/cart/add	Ajout au panier	USER
POST	/api/orders	Validation commande	USER
POST	/api/reviews	Depot d'avis	USER
GET	/api/reviews/product/{id}	Avis d'un produit	Public

Tableau 3 : Endpoints REST principaux

Chapitre 3

Choix techniques et justifications

3. Choix techniques et justifications

3.1. Backend - Java et Spring Boot

3.1.1. Pourquoi Java ?

Java demeure, en 2025, l'un des langages les plus largement deployes en entreprise - notamment dans le secteur bancaire, les telecommunications et le e-commerce de grande echelle. Ses atouts pedagogiques sont multiples : typage fort, programmation orientee objet stricte, ecosysteme mature, machine virtuelle performante. La version **Java 17 LTS** (Long-Term Support) a ete retenue pour sa stabilite et son adoption massive.

3.1.2. Pourquoi Spring Boot ?

Spring Boot est le framework Java de reference pour la construction d'applications backend modernes. Il automatise la configuration et fournit une myriade de starters prêts a l'emploi. Ses points forts sont :

- **Auto-configuration** : par convention plutot que par configuration explicite.
- **Embedded Tomcat** : deploiement en JAR auto-executable.
- **Spring Data** : abstraction puissante des SGBD.
- **Spring Security 6** : securisation standardisee.
- **Actuator** : observabilite prete a l'emploi.

3.2. Persistence hybride : MySQL + MongoDB

3.2.1. Justification du choix hybride

Le choix d'un **double systeme de persistence** n'est pas un caprice : il repond a une realite du terrain. Toutes les donnees ne sont pas egales :

- Les **transactions financieres** (commandes, panier, stock) exigent les proprietes ACID. Le SQL relationnel y excelle.
- Les **avis utilisateurs** ne necessitent ni jointures complexes, ni transactions ; ils croissent rapidement et sont consultes massivement en lecture. Le NoSQL documentaire les sert plus efficacement.

Critere	MySQL 8	MongoDB 7
Paradigme	SQL, tables normalisees	BSON, documents JSON
Schema	Strict, predefini	Flexible
Transactions	ACID natives	Limitees (multi-document v4+)
Joins	Performants	Rares (lookup)
Lectures massives	Bonnes (index)	Excellentes (replicas)
Cas E-Store	Users, Products, Orders	Reviews

Tableau 2 : Comparaison MySQL vs MongoDB

3.2.2. Bascule H2 / MySQL via les profils

Le projet exploite la fonctionnalité Spring de profils (*application-dev.properties*, *application-prod.properties*) pour basculer transparentement entre une base H2 in-memory (développement) et une base MySQL persistante (production).

```
# application-dev.properties
spring.datasource.url=jdbc:h2:mem:estore;MODE=MySQL
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

3.3. Frontend - Angular 17 standalone

3.3.1. Pourquoi Angular ?

Trois frameworks dominent le développement web SPA en 2025 : **Angular**, **React** et **Vue**. Angular se distingue par son approche tout-en-un (router, formulaires, HTTP, i18n inclus) et son langage TypeScript natif, qui apporte la rigueur du typage statique au JavaScript. Pour un projet pédagogique de fin de module, Angular offre l'environnement le plus structurant.

3.3.2. Le mode standalone d'Angular 17

Depuis la version 14, et de manière généralisée depuis la version 17, Angular permet de s'affranchir des *NgModule* historiques au profit de **composants standalone**. Avantages : code plus concis, lazy loading natif par route, imports explicites par composant, bundle initial plus léger.

3.3.3. Signals : la nouvelle reactivité

Les **signals** sont la grande nouveauté d'Angular 17. Ils remplacent avantageusement les Observable RxJS pour la gestion d'état simple. Dans E-Store, ils servent par exemple à afficher dynamiquement le badge du panier dans la barre de navigation :

```
@Injectable({ providedIn: 'root' })
export class CartService {
  readonly cart = signal<Cart | null>(null);
  readonly itemCount = signal<number>(0);

  add(productId: number, qty: number) {
    return this.http.post(...).pipe(
      tap(r => this.update(r.data))
    );
  }
}
```


Chapitre 4

Realisation du backend

4. Realisation du backend

4.1. Structure du projet

L'arborescence *estore-backend/src/main/java/com/estore/* reflète fidèlement le découpage en domaines :

```
com.estore/  
  EstoreApplication.java    <- Point d'entree  
  customer/                 {entity, dto, repository, service, controller, security}  
  catalog/                  {entity, dto, repository, service, controller}  
  inventory/                {entity, dto, repository, service, controller}  
  shopping/                 {entity, dto, repository, service, controller}  
  billing/                  {entity, dto, repository, service, controller}  
  review/                   {document, dto, repository, service, controller}  
  shared/                   ApiResponse.java  
  config/                   CorsConfig, SecurityConfig, DataSeeder  
  exception/                GlobalExceptionHandler + 3 exceptions métier
```

Chaque domaine est **autonome** : il ne peut dépendre que des couches inférieures de son propre domaine, ou des composants partagés (*shared*, *exception*). Les domaines communiquent exclusivement via leurs services publics.

4.2. Domaine customer

Ce domaine pivote autour de l'entité User, à laquelle est associé un Profile en relation @OneToOne :

```
@Entity  
@Table(name = "users")  
public class User {  
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(nullable = false, unique = true)  
    private String email;  
  
    @JsonIgnore  
    @Column(nullable = false)  
    private String password;           // hash BCrypt  
  
    @Enumerated(EnumType.STRING)  
    private Role role;                // USER ou ADMIN  
  
    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL,  
               orphanRemoval = true, fetch = FetchType.LAZY)  
    private Profile profile;  
}
```

L'annotation @JsonIgnore sur password empêche toute sérialisation accidentelle. La cascade ALL sur profile garantit l'atomicité création/suppression.

4.3. Domaine catalog

Le catalogue regroupe Category et Product. Le repository ProductRepository expose une méthode personnalisée combinant recherche textuelle et filtre par catégorie :

```
public interface ProductRepository extends JpaRepository<Product, Long> {  
    @Query("""
```

```

        SELECT p FROM Product p
        WHERE (:categoryId IS NULL OR p.category.id = :categoryId)
            AND (:q IS NULL OR LOWER(p.name) LIKE LOWER(CONCAT('%',:q,'%'))
                OR LOWER(p.description) LIKE LOWER(CONCAT('%',:q,'%')))
        """)
    Page<Product> search(@Param("categoryId") Long categoryId,
                        @Param("q") String q, Pageable pageable);
}

```

L'utilisation de JPQL plutot que de SQL natif garantit la portabilite H2/MySQL, et la pagination via Pageable evite le retour massif d'enregistrements.

4.4. Domaines shopping et billing

4.4.1. Le panier (shopping)

L'entite Cart maintient une relation @OneToMany avec CartItem. A l'ajout d'un produit deja present, la quantite est incrementee plutot que dupquee.

4.4.2. Le checkout transactionnel (billing)

La validation de commande constitue **l'operation la plus critique** de l'application. Elle touche trois agregats (Cart, Order, Inventory) et doit etre strictement atomique :

```

@Transactional
public OrderDto checkout() {
    Cart cart = cartService.getOrCreateCart(user);
    if (cart.getItems().isEmpty())
        throw new BusinessException("Votre panier est vide");

    // 1) Verifier le stock pour TOUS les items AVANT modification
    for (CartItem ci : cart.getItems())
        inventoryService.checkAvailability(
            ci.getProduct().getId(), ci.getQuantity());

    // 2) Creer la commande + decrementer le stock
    Order order = Order.builder().user(user)
        .status(OrderStatus.PENDING).build();
    BigDecimal total = BigDecimal.ZERO;
    for (CartItem ci : cart.getItems()) {
        order.getItems().add(OrderItem.builder()...build());
        total = total.add(ci.getUnitPrice().multiply(...));
        inventoryService.decrement(ci.getProduct().getId(),
            ci.getQuantity());
    }
    order.setTotalAmount(total);
    order.setStatus(OrderStatus.CONFIRMED);

    // 3) Sauvegarder + vider le panier
    Order saved = orderRepository.save(order);
    cartService.clearCart(cart);
    return OrderDto.from(saved);
}

```

L'annotation @Transactional de Spring garantit qu'en cas d'exception levee a n'importe quelle etape, toutes les modifications faites depuis l'entree dans la methode sont automatiquement annulees (*rollback*). C'est l'**atomicite ACID** en action.

4.5. Domaine review (MongoDB)

Le document Review est mappe sur la collection MongoDB :

```
@Document(collection = "reviews")
public class Review {
    @Id private String id;
    @Indexed private Long productId;
    @Indexed private Long userId;
    private String authorName;
    private int rating;
    private String comment;
    private Instant createdAt;
}
```

Le repository tire profit de la convention over configuration de Spring Data : la simple declaration d'une methode au nom evocateur produit la requete correspondante.

```
public interface ReviewRepository
    extends MongoRepository<Review, String> {
    List<Review> findByProductIdOrderByCreatedAtDesc(Long productId);
    List<Review> findByUserIdOrderByCreatedAtDesc(Long userId);
}
```

4.6. Gestion globale des erreurs

Plutot que de laisser remonter des exceptions Java brutes vers le client, l'application centralise leur traitement via un @RestControllerAdvice :

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<Void>> handleNotFound(
        ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ApiResponse.error(ex.getMessage()));
    }

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ApiResponse<Void>> handleBusiness(
        BusinessException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(ApiResponse.error(ex.getMessage()));
    }
}
```


Chapitre 5

Securite de l'application

5. Securite de l'application

La securite de toute application e-commerce est une exigence absolue. E-Store met en oeuvre une chaine complete depuis le hachage des mots de passe jusqu'a l'autorisation fine par role.

5.1. Hachage BCrypt des mots de passe

Aucun mot de passe utilisateur n'est jamais stocke en clair. Tous transitent par l'algorithme **BCrypt**, un one-way hash avec sel aleatoire et facteur de cout ajustable. Le facteur 10 par default produit 1024 rounds de calcul, rendant tout brute-force prohibitif.

5.2. Authentification stateless par JWT

L'authentification suit un modele *stateless* : aucun etat n'est conserve cote serveur entre deux requetes. Le client porte lui-meme la preuve de son identite, sous la forme d'un **JSON Web Token** signe HMAC-SHA256.

5.2.1. Structure d'un JWT

Un JWT est compose de trois parties separees par un point : header.payload.signature. Le payload de E-Store contient les claims suivants :

- **sub** : email de l'utilisateur (subject)
- **uid** : identifiant numerique
- **role** : USER ou ADMIN
- **name** : nom complet (pour affichage UI)
- **iat** et **exp** : timestamps de creation et d'expiration

5.2.2. Cycle de vie complet

#	Etape	Acteur
1	Login : POST /api/auth/login (email, password)	Client
2	AuthService valide via passwordEncoder.matches()	Serveur
3	JwtService.generateToken() - HS256, exp 24h	Serveur
4	Client persiste le token dans localStorage	Client
5	Toute requete : header Authorization: Bearer <token>	Client
6	JwtAuthenticationFilter valide signature et expiration	Serveur
7	SecurityContext.setAuthentication(...)	Serveur

Figure 5 : Cycle de vie d'un JWT

5.2.3. Generation du token

```
public String generateToken(User user) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("uid", user.getId());
    claims.put("role", user.getRole().name());
    claims.put("name", user.getFirstName() + " " + user.getLastName());

    Date now = new Date();
    Date exp = new Date(now.getTime() + expirationMs); // 24h

    return Jwts.builder()
        .claims(claims)
        .subject(user.getEmail())
        .issuedAt(now).expiration(exp)
        .signWith(key) // HMAC-SHA256
        .compact();
}
```

5.3. Autorisation par role

Les operations sensibles (CRUD categories/produits/stock) sont protegees par `@PreAuthorize` :

```
@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<ApiResponse<ProductDto>> create(
    @Valid @RequestBody CreateProductDto dto) {
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(ApiResponse.ok("Produit cree", productService.create(dto)));
}
```

Tout appel par un utilisateur non-ADMIN se solde par un HTTP 403 Forbidden, intercepte par le `GlobalExceptionHandler`.

5.4. Configuration CORS

En developpement, le frontend tourne sur le port 4200 et le backend sur le port 8080. Cette difference d'origine declencherait une erreur CORS du navigateur sans configuration appropriee :

```
@Override
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
        .allowedOrigins("http://localhost:4200")
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("*").allowCredentials(true);
}
```


Chapitre 6

Realisation du frontend

6. Realisation du frontend

6.1. Structure de l'application Angular

Le projet Angular suit l'organisation **core / shared / features**, classique des applications Angular modernes :

```
src/app/
  app.component.ts      <- Composant racine
  app.config.ts         <- Configuration globale
  app.routes.ts         <- Routes
  core/
    services/           <- AuthService, CartService, ...
    guards/             <- AuthGuard
    interceptors/       <- AuthInterceptor, ErrorInterceptor
    models/             <- Interfaces TypeScript
  shared/
    components/         <- Header, Footer, Loader, Toast
  features/
    auth/               <- Login, Register
    catalog/            <- ProductList, ProductDetail
    cart/               <- Cart
    orders/             <- Orders
    profile/            <- Profile
    reviews/           <- ReviewForm
```

6.2. Routing et lazy loading

Toutes les routes utilisent *loadComponent* pour un lazy loading natif, ce qui produit des bundles separees par fonctionnalite :

```
export const routes: Routes = [
  { path: '', loadComponent: () =>
    import('./features/catalog/product-list.component')
      .then(m => m.ProductListComponent) },
  { path: 'login', loadComponent: () =>
    import('./features/auth/login.component')
      .then(m => m.LoginComponent) },
  { path: 'cart', loadComponent: () =>
    import('./features/cart/cart.component')
      .then(m => m.CartComponent),
    canActivate: [authGuard] },
];
```

Mesure : le bundle initial pese 13.84 ko, et chaque feature ajoute en moyenne 8 ko charges a la demande.

6.3. AuthInterceptor et ErrorInterceptor

L'AuthInterceptor attache silencieusement le JWT a chaque requete HTTP sortante :

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token;
  if (!token) return next(req);
  return next(req.clone({
    setHeaders: { Authorization: `Bearer ${token}` }
  }));
};
```

L'ErrorInterceptor centralise la gestion d'erreurs : 401 entraine la deconnexion automatique, 4xx affichent un toast :

```
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const toast = inject(ToastService);
  const auth = inject(AuthService);
  const router = inject(Router);

  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) {
        auth.logout();
        router.navigate(['/login']);
        toast.error('Session expiree');
      }
      return throwError(() => err);
    })
  );
};
```

6.4. AuthGuard

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isAuthenticated) return true;
  router.navigate(['/login']);
  return false;
};
```

Trois routes sont protegees par cette garde : */cart*, */orders* et */profile*.

Chapitre 7

Tests et validation

7. Tests et validation

7.1. Strategie de tests

Suivant la **pyramide de tests** de Mike Cohn, nous avons concentre l'effort sur la base de la pyramide : les tests unitaires des services metier. Ces tests sont rapides, deterministes et faciles a maintenir.

Outils utilises :

- **JUnit 5** (Jupiter) : framework de test
- **Mockito 5** : isolation des dependances
- **AssertJ** : assertions fluentes (assertThat...)
- **Spring Boot Test** : pour les tests d'integration

7.2. Suites de tests implementees

Suite	Tests	Statut	Couverture
ProductServiceTest	4	PASS	findById, recherche par mot-cle
CartServiceTest	2	PASS	Ajout avec stock suffisant/insuffisant
OrderServiceTest	3	PASS	Panier vide, validation, historique
TOTAL	9	0 failure	BUILD SUCCESS

Tableau 5 : Tests unitaires implementes

7.3. Exemple de test

```
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
class OrderServiceTest {

    @Mock private OrderRepository orderRepository;
    @Mock private CartService cartService;
    @Mock private InventoryService inventoryService;
    @InjectMocks private OrderService orderService;

    @Test
    void checkout_panierValide_creeCommandeEtViderPanier() {
        cart.getItems().add(item);
        when(orderRepository.save(any(Order.class)))
            .thenAnswer(inv -> { Order o = inv.getArgument(0);
                                o.setId(500L); return o; });

        OrderDto dto = orderService.checkout();

        assertThat(dto.getStatus()).isEqualTo(OrderStatus.CONFIRMED);
        assertThat(dto.getTotalAmount())
            .isEqualToComparingTo(new BigDecimal("25000.00"));
        verify(inventoryService).decrement(10L, 2);
        verify(cartService).clearCart(cart);
    }

    @Test
    void checkout_panierVide_lanceException() {
        assertThatThrownBy(() -> orderService.checkout())
            .isInstanceOf(BusinessException.class)
            .hasMessageContaining("vide");
    }
}
```

7.4. Resultats d'execution

```
$ ./mvnw test
[INFO] Tests run: 3, Failures: 0, Errors: 0 -- OrderServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0 -- ProductServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0 -- CartServiceTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```


Chapitre 8

Deploiement et DevOps

8. Deploiement et DevOps

8.1. Profils Spring

L'application fournit deux profils d'execution :

- **dev** (par default) : H2 in-memory, MongoDB optionnel. Demarrage instantane sans installation prealable.
- **prod** : MySQL 8 + MongoDB 7, accessibles via Docker Compose.

8.2. Conteneurisation Docker

Le fichier docker-compose.yml orchestre quatre services :

```
services:
  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: estore
    ports: ["3306:3306"]
    volumes: [mysql-data:/var/lib/mysql]

  mongo:
    image: mongo:7
    ports: ["27017:27017"]
    volumes: [mongo-data:/data/db]

  phpmyadmin:
    image: phpmyadmin:latest
    ports: ["8081:80"]

  mongo-express:
    image: mongo-express:latest
    ports: ["8082:8081"]
```

Une commande suffit a provisionner l'environnement :

```
$ docker compose up -d
```

8.3. Demarrage en trois commandes

```
# 1. Bases de donnees (optionnel : profil prod uniquement)
docker compose up -d

# 2. Backend
cd estore-backend && ./mvnw spring-boot:run

# 3. Frontend (autre terminal)
cd estore-frontend && npm install && npm start

# 4. Ouvrir http://localhost:4200
```

8.4. Versionnement Git et publication GitHub

L'historique du projet est versionne par Git, organise en **21 commits** semantiques (feat, fix, docs, chore). Le projet est publie sur GitHub a l'adresse : github.com/akrambelmoussa-etu-byte/e-store

Chapitre 9

Difficultes rencontrees et solutions

9. Difficultes rencontrees et solutions

Le developpement n'a pas ete un long fleuve tranquille. Plusieurs obstacles techniques significatifs ont ete rencontres ; les surmonter a ete l'occasion d'approfondir notre comprehension de l'ecosysteme.

9.1. Strictness Mockito

9.1.1. Symptome

Lors du premier lancement de *mvn test*, deux tests echouent avec le message :

```
UnnecessaryStubbingException: Unnecessary stubbings detected.
```

9.1.2. Cause

Mockito 5 applique par default le mode strict. Toute instruction *when(...).thenReturn(...)* declaree mais non verifiee durant l'execution du test provoque une erreur. Or, nos methodes *@BeforeEach* configuraient des stubs partages entre plusieurs tests.

9.1.3. Solution

Annoter chaque classe de test avec :

```
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
class CartServiceTest { ... }
```

9.2. LazyInitializationException

9.2.1. Symptome

Lors du retour d'une entite User via le controleur, Hibernate levait une LazyInitializationException sur le Profile charge en mode LAZY.

9.2.2. Cause

La serialisation Jackson tentait de naviguer dans la relation apres la fermeture de la session JPA.

9.2.3. Solution

Adoption systematique du pattern **DTO** : chaque endpoint renvoie un objet de transfert (UserDetailsDto), construit dans la couche service alors que la session est encore ouverte.

9.3. Installation de Docker Desktop

9.3.1. Symptome

Lors de l'installation sous Windows 11, l'erreur suivante bloque le processus :

```
For security reasons C:\ProgramData\DockerDesktop must
be owned by an elevated account.
```

9.3.2. Cause

Un dossier residuel d'une installation anterieure subsistait avec des permissions incorrectes.

9.3.3. Solutions implementees

Solution A - Brute : suppression manuelle en mode administrateur :

```
rmdir /s /q "C:\ProgramData\DockerDesktop"  
rmdir /s /q "C:\ProgramData\Docker"
```

Solution B - Elegante : mise en place d'un profil dev base sur H2 in-memory, qui permet de demarrer l'application sans aucune installation prealable. Cette dualite Docker/H2 a ete pensee des la conception.

9.4. Encodage des tokens JWT

9.4.1. Symptome

Lors de la generation du premier JWT, la bibliotheque *jjwt* levait une *WeakKeyException* : la cle secrete configuree comportait moins de 256 bits.

9.4.2. Solution

Implementer une logique de fallback : tenter d'abord un decodage Base64 de la cle, et en cas d'echec ou de longueur insuffisante, utiliser l'encodage UTF-8 brut :

```
byte[] bytes;  
try {  
    bytes = Decoders.BASE64.decode(secret);  
    if (bytes.length < 32) throw new IllegalArgumentException();  
} catch (Exception e) {  
    bytes = secret.getBytes(StandardCharsets.UTF_8);  
}  
this.key = Keys.hmacShaKeyFor(bytes);
```


Chapitre 10

Resultats et perspectives

10. Resultats et demonstration

10.1. Bilan fonctionnel

L'application livree couvre integralement le perimetre du cahier des charges :

Fonctionnalite	Description	Statut
Inscription	Email + mot de passe, JWT 24h	Livre
Connexion	Authentification stateless	Livre
Profil utilisateur	Consultation et modification	Livre
Catalogue paginer	Recherche, filtre par categorie	Livre
Fiche produit	Detail, image, stock, avis	Livre
Panier persistant	Ajout, modification, vidage	Livre
Validation commande	Transactionnelle ACID	Livre
Historique commandes	Consultation detaillee	Livre
Avis produits	Depot et consultation (MongoDB)	Livre
Administration	CRUD categories/produits/stock	Livre

Tableau 7 : Bilan fonctionnel

10.2. Metriques de qualite

Indicateur	Valeur
Lignes de code Java	~ 3 500
Lignes de code TypeScript	~ 2 000
Nombre de classes Java	71
Nombre de composants Angular	14
Nombre d'endpoints REST	24
Tests unitaires	9 (100% passants)
Bundle JS initial	13.84 ko
Temps de demarrage backend (H2)	~ 4 s
Temps de reponse API (mediane)	~ 30 ms
Commits Git	21

Tableau 6 : Metriques quantitatives du projet

10.3. Captures d'ecran de la demonstration

Les captures d'ecran de l'application en fonctionnement sont disponibles dans le dossier *docs/screenshots/* du depot GitHub. Le scenario de demonstration suit huit etapes :

- 1. Page catalogue avec 12 produits seed
- 2. Filtrage par categorie 'Sport'
- 3. Fiche detaillee d'un produit
- 4. Tentative d'ajout au panier non authentifie -> redirection
- 5. Connexion avec le compte de test
- 6. Ajout de deux produits au panier
- 7. Validation de la commande, redirection /orders
- 8. Depot d'un avis 5 etoiles, persistance MongoDB

Conclusion generale

Bilan

Au terme de ce projet, nous avons mene a bien la conception et la realisation d'une application e-commerce **integralement fonctionnelle**, couvrant le parcours utilisateur du visiteur anonyme jusqu'a la commande validee. L'architecture en **trois couches x cinq domaines** a ete rigoureusement respectee, fournissant une base saine pour toute evolution ulterieure.

Sur le plan technique, les objectifs initiaux ont tous ete atteints :

- Architecture full-stack moderne avec Spring Boot 3.3 et Angular 17.
- Persistance hybride MySQL + MongoDB operationnelle.
- Securite robuste par BCrypt + JWT + @PreAuthorize.
- Suite de tests JUnit 5 / Mockito en BUILD SUCCESS.
- Conteneurisation Docker Compose, profil de fallback H2.

Sur le plan humain, le travail en binome nous a permis d'expérimenter *in vivo* les pratiques du developpement collaboratif : versionnement Git, conventions de commit, revue de code, repartition des taches, communication asynchrone. Ces competences relationnelles, souvent sous-estimees, sont determinantes en milieu professionnel.

Apports personnels

Au-dela de la technique, ce projet nous a confronte a la **realite de l'incertitude** qui caracterise tout developpement logiciel : choix d'architecture aux consequences durables, debogage de problemes inattendus (la mesaventure Docker en est l'archetype), arbitrages entre elegance du code et delais. Apprendre a composer avec cette incertitude, plutot qu'a la fuir, est probablement le benefice le plus durable de cette experience.

Perspectives

Le projet livre n'est pas un produit fini mais un **socle evolutif**. Plusieurs pistes d'extension ont ete identifiees :

- **Paie ment en ligne** : integration de Stripe ou des passerelles marocaines (CMI, Maroc Telecommerce).
- **Notifications transactionnelles** : envoi automatique d'emails (Spring Mail) ou de SMS (Twilio).
- **Recommandations** : suggestion produits basee sur l'historique d'achat (collaborative filtering).
- **Internationalisation** : passage du francais a l'arabe ou l'anglais via Angular i18n.
- **Deploiement cloud** : conteneurisation et deploiement sur Heroku, Render ou AWS Free Tier.

- **Application mobile** : portage Ionic ou Flutter consommant la meme API.
- **Observabilite** : integration Prometheus + Grafana pour les metriques applicatives.

Plus fondamentalement, l'architecture en couches et en domaines que nous avons mise en place rend ces evolutions **additives** : il sera possible de les introduire sans casser l'existant. C'est, a nos yeux, la preuve la plus convaincante que les choix architecturaux de depart etaient les bons.

Bibliographie et webographie

Ouvrages

- [1] Craig Walls, *Spring Boot in Action*, 2nd edition, Manning Publications, 2024.
- [2] Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.
- [3] Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- [4] Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

Documentations officielles

- [5] Spring Framework : <https://docs.spring.io/spring-framework/reference/>
- [6] Spring Boot : <https://docs.spring.io/spring-boot/>
- [7] Spring Data JPA : <https://docs.spring.io/spring-data/jpa/reference/>
- [8] Spring Data MongoDB : <https://docs.spring.io/spring-data/mongodb/reference/>
- [9] Angular Documentation : <https://angular.dev/>
- [10] MongoDB Manual : <https://www.mongodb.com/docs/manual/>
- [11] RFC 7519 (JWT) : <https://www.rfc-editor.org/rfc/rfc7519>
- [12] OWASP Authentication : <https://cheatsheetseries.owasp.org/>

Articles et tutoriels

- [13] Baeldung, 'A Guide to JPA with Spring', baeldung.com/the-persistence-layer-with-spring-and-jpa
- [14] Baeldung, 'Spring Security with JWT', baeldung.com/spring-security-oauth-jwt
- [15] Angular Blog, 'Introducing Angular Signals', blog.angular.io

Annexes

Annexe A : Configuration application.properties

```
# === application.properties (commun) ===
spring.application.name=estore-backend
spring.profiles.active=dev
server.port=8080

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false

# MongoDB
spring.data.mongodb.uri=mongodb://localhost:27017/estore

# JWT
jwt.secret=change-me-with-a-256-bits-secret-key-for-production
jwt.expiration=86400000

# === application-dev.properties (H2 in-memory) ===
spring.datasource.url=jdbc:h2:mem:estore;MODE=MySQL
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# === application-prod.properties (MySQL) ===
spring.datasource.url=jdbc:mysql://localhost:3306/estore?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=root
```

Annexe B : Comptes de test

Le DataSeeder cree automatiquement deux comptes au premier demarrage si la base est vide :

Role	Email	Mot de passe
ADMIN	admin@estore.ma	Admin@123
USER	user@estore.ma	User@123

Annexe C : Procedure de demarrage rapide

```
# 1. Cloner le depot
git clone https://github.com/akrambelmoussa-et-byte/e-store.git
cd e-store

# 2a. Demarrer en mode dev (H2 + Mongo optionnel)
cd estore-backend
./mvnw spring-boot:run

# 2b. (Alternative) Demarrer en mode prod
docker compose up -d
cd estore-backend
./mvnw spring-boot:run -Dspring.profiles.active=prod

# 3. Frontend (terminal separe)
cd estore-frontend
npm install
npm start

# 4. Ouvrir http://localhost:4200
```

Annexe D : Structure de l'arborescence

```
e-store/
  README.md
  docker-compose.yml
  estore-backend/                                Spring Boot + Maven
    src/main/java/com/estore/
      EstoreApplication.java
      customer/    catalog/    inventory/
      shopping/    billing/    review/
      shared/      config/     exception/
    src/main/resources/
      application.properties
      application-dev.properties
      application-prod.properties
    src/test/java/com/estore/                    9 tests JUnit 5
  estore-frontend/                               Angular 17
    src/app/
      core/          (services, guards, interceptors)
      shared/        (header, footer, loader, toast)
      features/      (auth, catalog, cart, orders, profile, reviews)
  docs/              PDFs explicatifs
  rapport-latex/     Sources LaTeX du rapport
```